

TZI

STÁTNICE

TZI

Teoretické základy informatiky

Matematické, algoritmické, kryptografické a virtualizační základy

Obsah všech otázek

01	Základní pojmy teorie čísel: prvočíslo, dělitelnost, NSD
02	Konečná tělesa a principy modulární aritmetiky
03	Moderní metody šifrování, elektronický podpis a metoda RSA
04	Důkazové techniky: matematická indukce a princip inkluze a exkluze
05	Algoritmus jako výpočetní model a Churchova teze
06	Rozhodnutelné, nerozhodnutelné, částečně rozhodnutelné problémy a vyčíslitelné funkce
07	Výpočetní složitost problémů, redukce a polynomiální redukce
08	Úplné problémy z hlediska rozhodnutelnosti, NP-úplné problémy
09	Blockchain technologie a kybernetická bezpečnost
10	Distribuované a paralelní souborové systémy, NFS a GFS
11	Virtualizace: Druhy, hardwarová akcelerace a přístup k hardwaru
12	Stínové stránkování a migrace virtuálních strojů
13	Zajištění vysoké dostupnosti a Virtual Network Embedding
14	SDN, NFV, SDWAN a VXLAN
15	Cloud Computing: Distribuční a platební modely, základní pojmy

Základní pojmy teorie čísel: prvočíslo, dělitelnost, NSD

Vysvětlete základní pojmy teorie čísel, prvočíslo, dělitelnost. Algoritmy a věty o největším společném děliteli.

Dělitelnost

Definice

Celočíselné dělení beze zbytku. Celé číslo a dělí celé číslo b (značíme $a \mid b$), pokud existuje celé číslo k takové, že $b = k \cdot a$.

Vlastnosti

- Reflexivita: $a \mid a$
- Tranzitivita: Pokud $a \mid b$ a $b \mid c$, pak $a \mid c$
- Pokud $a \mid b$ a $a \mid c$, pak $a \mid (b + c)$ a $a \mid (b - c)$

Prvočíslo a složené číslo

Prvočíslo

Přirozené číslo $p > 1$, které má přesně dva kladné dělitele: 1 a p .

Složené číslo

Přirozené číslo > 1 , které má alespoň tři různé kladné dělitele (lze zapsat jako součin dvou menších přirozených čísel > 1).

Základní věta aritmetiky

Každé přirozené číslo $n > 1$ lze jednoznačně rozložit na součin prvočísel (až na pořadí činitelů).

- $n = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdot \dots \cdot p_k^{\alpha_k}$

Největší společný dělitel (NSD / GCD)

Definice

Nechť $a, b \in \mathbb{Z}$. Největší společný dělitel $\text{NSD}(a, b)$ je největší přirozené číslo d , pro které platí $d \mid a$ a zároveň $d \mid b$.

Nesoudělnost

Čísla a, b jsou nesoudělná, pokud $\text{NSD}(a, b) = 1$.

Algoritmy a věty o NSD

Euklidův algoritmus

Účel

Rychlé nalezení NSD dvou čísel bez nutnosti jejich rozkladu na prvočísla.

Princip

Využívá faktu, že $\text{NSD}(a, b) = \text{NSD}(b, a \bmod b)$.

Postup

1. Máme $a \geq b > 0$.
2. Spočítáme zbytek $r = a \bmod b$.
3. Pokud $r = 0$, pak b je hledaný NSD.
4. Pokud $r \neq 0$, přiřadíme $a \leftarrow b, b \leftarrow r$ a jdeme na krok 2.

Složitost

$O(\log(\min(a, b)))$ - velmi efektivní.

Rozšířený Euklidův algoritmus

Účel

Nachází NSD a zároveň koeficienty (tzv. Bézoutovy koeficienty) x, y pro tzv. **Bézoutovu rovnost**:
 $\text{NSD}(a, b) = a \cdot x + b \cdot y$.

Využití

Modulární inverze v kryptografii (např. v RSA).

Nejmenší společný násobek (NSN / LCM)

Definice

Nejmenší kladné číslo m , které je dělitelné a i b .

Vztah k NSD

$$a \cdot b = \text{NSD}(a, b) \cdot \text{NSN}(a, b).$$

Praktické příklady

- 1. Synchronizace intervalů:** Dva procesy se spouštějí každých 84 s a 30 s. Jejich společný rytmus určíme přes $\text{NSD}(84, 30) = 6$ a $\text{NSN}(84, 30) = \frac{84 \cdot 30}{6} = 420$, takže se oba znovu spustí současně každých 420 sekund.
- 2. Modulární inverze v RSA:** Když má RSA $\phi(n) = 40$ a veřejný exponent $e = 7$, hledáme d tak, aby $7d \equiv 1 \pmod{40}$. Rozšířený Euklidův algoritmus dá $7 \cdot 23 - 4 \cdot 40 = 1$, takže soukromý exponent je $d = 23$.

Konečná tělesa a principy modulární aritmetiky

Konečná tělesa a principy modulární aritmetiky.

Modulární aritmetika

Princip

Aritmetika se zbytkem po dělení. Pracujeme s čísly v určitém rozsahu (modulu). "Hodinová aritmetika".

Kongruence

Značíme $a \equiv b \pmod{n}$, což znamená, že čísla a a b dávají po dělení n stejný zbytek.

- Ekvivalentní definice: $n \mid (a - b)$.

Základní operace

- Sčítání: $(a + b) \pmod{n} = ((a \pmod{n}) + (b \pmod{n})) \pmod{n}$
- Odčítání: $(a - b) \pmod{n} = ((a \pmod{n}) - (b \pmod{n})) \pmod{n}$
- Násobení: $(a \cdot b) \pmod{n} = ((a \pmod{n}) \cdot (b \pmod{n})) \pmod{n}$

Inverzní prvek (dělení v modulární aritmetice)

- Hledáme číslo x , pro které platí $a \cdot x \equiv 1 \pmod{n}$.
- Značíme $x = a^{-1} \pmod{n}$.

Podmínka existence

Inverzní prvek existuje právě tehdy, když jsou a a n **nesoudělná** ($\text{NSD}(a, n) = 1$).

- K hledání se používá **Rozšířený Euklidův algoritmus**.

Konečná tělesa (Galoisova tělesa)

Těleso (Field)

Algebraická struktura množiny F s operacemi $+$ a $\cdot$, kde funguje komutativita, asociativita a distributivita, a existují neutrální i inverzní prvky pro obě operace (kromě inverze k nule pro násobení). Podobně jako množina reálných čísel $\mathbb{R}$.

Konečné těleso

Těleso, které má konečný počet prvků (řád tělesa). Značí se $GF(q)$ (Galois Field).

Vlastnosti řádu tělesa

- Konečné těleso $GF(q)$ existuje pouze pokud $q = p^k$, kde p je prvočíslo a k je přirozené číslo.
- Tělesa stejného řádu jsou izomorfní (až na přeznačení prvků identická).

$GF(p)$ – Těleso prvočíselného řádu

- Nejjednodušší případ konečného tělesa (kde $k = 1$).
- Prvky jsou $\{0, 1, \dots, p - 1\}$.
- Operace $+$ a $\cdot$ se provádějí $(\text{mod } p)$.
- Protože p je prvočíslo, každý nenulový prvek je s p nesoudělný, takže má inverzi. Množina tvoří těleso.

$GF(p^k)$ – Těleso složeného řádu (Rozšíření tělesa)

- Např. $GF(2^8)$, fundamentální v IT a kryptografii (AES šifra), protože prvky perfektně odpovídají 1 bytu.
- Prvky nereprezentují běžná čísla, ale **polynomy s koeficienty v $GF(p)$** stupně ostře menšího než k .

Sčítání

Probíhá jako sčítání koeficientů odpovídajících mocnin modulo p (v $GF(2)$ jde o bitovou operaci XOR).

Násobení

Polynomy se násobí běžně, ale výsledek se navíc redukuje **modulo ireducibilní polynom** stupně k (polynom, který nelze rozložit, obdoba prvočísla u polynomů).

Praktické příklady

1. **Hodinová aritmetika:** Pokud je 10 hodin a přičteme 5 hodin, běžně řekneme 3 hodiny, protože $10 + 5 \equiv 3 \pmod{12}$. Stejný princip používá počítač při práci s omezeným rozsahem hodnot.
2. **AES a $GF(2^8)$:** V AES se jeden byte chápe jako prvek konečného tělesa $GF(2^8)$. Sčítání dvou bytů je XOR, takže například $0x57 \text{ XOR } 0x83 = 0xD4$; násobení už probíhá jako násobení polynomů s redukcí modulo pevně daný ireducibilní polynom.

Moderní metody šifrování, elektronický podpis a metoda RSA

Popište moderní metody šifrování, veřejné kódy, elektronický podpis Popište šifrování s veřejným klíčem – metoda RSA.

Moderní metody šifrování

Moderní kryptografie se rozděluje na dvě hlavní větve podle způsobu práce s klíči.

1. Symetrické šifrování (Kryptografie tajného klíče)

Princip

Používá se stejný klíč pro šifrování i dešifrování. Obě strany (odesílatel, příjemce) musí tento klíč sdílet v tajnosti.

Výhody

Velmi rychlé zpracování dat. Vhodné pro velké objemy dat.

Nevýhody

Problém distribuce klíčů (jak si jej bezpečně předat) a správa klíčů při velkém počtu účastníků.

Typické algoritmy

AES (Advanced Encryption Standard), DES/3DES, ChaCha20.

2. Asymetrické šifrování (Kryptografie veřejného klíče)

Princip

Každý uživatel má dvojici klíčů (tzv. asymetrický pár):

Veřejný klíč (Public key)

Může ho znát kdokoli, slouží k zašifrování zprávy pro daného uživatele (nebo k ověření jeho podpisu).

Soukromý klíč (Private key)

Musí zůstat přísně utajen, slouží k dešifrování zprávy (nebo k vytvoření vlastního podpisu).

Matematický základ

Bezpečnost stojí na **jednosměrných funkcích** s padacími dvířky (trapdoor). Z veřejného klíče nelze efektivně (v rozumném čase) spočítat ten soukromý.

Výhody

Řeší problém bezpečné distribuce klíčů napříč nezabezpečeným kanálem.

Nevýhody

Výpočetně náročné (pomalé). V praxi se využívá **hybridní šifrování** (asymetricky se vymění bezpečný dočasný klíč, symetricky se přenáší data).

Elektronický podpis

Účel

Zajišťuje **autenticitu** (identita autora), **integritu** (data nebyla změněna) a **nepopiratelnost** (autor nemůže zapřít, že podepsal).

Princip fungování

1. Odesílatel aplikuje hashovací funkci (např. SHA-256) na posílaná data -> vznikne otisk (hash).
2. Odesílatel tento otisk "zašifruje" svým **soukromým klíčem**. Výsledkem je elektronický podpis (přiloží se ke zprávě).
3. Příjemce obdrží zprávu a podpis. Zprávu sám zahašuje.
4. Příjemce "dešifruje" podpis pomocí **veřejného klíče** odesílatele a získá otisk H_1 . Vlastní otisk označíme jako H_2 .
5. Pokud se $H_1 = H_2$, je podpis platný.

Metoda RSA (Rivest-Shamir-Adleman)

Nejpoužívanější asymetrický algoritmus, založený na obtížnosti faktorizace (rozkladu velkých čísel na prvočísla).

Generování klíčů

1. Zvolíme dvě velká, náhodná prvočísla p a q .
2. Vypočítáme modul $n = p \cdot q$. (Číslo n je součástí veřejného i soukromého klíče).
3. Vypočítáme Eulerovu funkci $\phi(n) = (p - 1)(q - 1)$. (Odpovídá počtu čísel $\leq n$ nesoudělných s n).
4. Zvolíme veřejný exponent e takový, že $1 < e < \phi(n)$ a $\text{NSD}(e, \phi(n)) = 1$. (Často např. 65537).
5. Vypočítáme soukromý exponent d , což je modulární inverze e v $(\text{mod } \phi(n))$. Tedy: $d \cdot e \equiv 1 \pmod{\phi(n)}$

Veřejný klíč

(n, e)

Soukromý klíč

(d) (a případně parametry p, q)

Šifrování a dešifrování

Zpráva m

se převede na číslo $0 \leq m < n$.

Šifrování

Odesílatel použije veřejný klíč (n, e) : $c = m^e \bmod n$

Dešifrování

Příjemce použije soukromý klíč d : $m = c^d \bmod n$

Praktické příklady

- 1. HTTPS v prohlížeči:** Při připojení k bankovnímu webu se asymetrická kryptografie použije k bezpečnému dohodnutí klíče a následná komunikace už běží rychle symetricky, typicky přes AES. Prakticky se tím spojuje bezpečná výměna klíče s vysokou rychlostí přenosu.
- 2. Mini RSA výpočet:** Zvolíme $p = 5$, $q = 11$, tedy $n = 55$ a $\phi(n) = 40$. Pro $e = 3$ vyjde $d = 27$, protože $3 \cdot 27 \equiv 1 \pmod{40}$. Zpráva $m = 7$ se zašifruje jako $c = 7^3 \bmod 55 = 13$ a dešifrováním $13^{27} \bmod 55$ dostaneme zpět 7.

Důkazové techniky: matematická indukce a princip inkluze a exkluze

Důkazové techniky matematická indukce a princip inkluze a exkluze, aplikace na příklady.

Matematická indukce

Účel

Slouží k dokazování tvrzení o přirozených číslech (např. že určitý vzorec platí pro všechna $n \in \mathbb{N}$).

Princip (Domino efekt)

Důkaz se rozděluje na dva nezbytné kroky:

Základní krok (báze indukce)

Dokážeme, že tvrzení platí pro nejmenší možné číslo (obvykle $n = 1$ nebo $n = 0$). To odpovídá poražení první kostky domina.

Indukční krok

Provedeme **indukční předpoklad**, že tvrzení platí pro nějaké obecné $n = k$. Z tohoto předpokladu matematicky odvodíme, že tvrzení logicky musí platit i pro následníka, tj. pro $n = k + 1$. To je důkaz faktu: "pokud spadne k -tá kostka domina, zaručeně shodí $(k + 1)$ -ní".

Aplikace na příklad (Matematická indukce)

Tvrzení: Součet prvních n přirozených čísel je $\frac{n(n+1)}{2}$.

Báze ($n = 1$)

Levá strana: 1. Pravá strana: $\frac{1(1+1)}{2} = 1$. Tvrzení pro $n = 1$ platí.

Indukční předpoklad

Platí pro k : $1 + 2 + \dots + k = \frac{k(k+1)}{2}$.

Indukční krok (dokázat pro $k + 1$)

Chceme dokázat $1 + 2 + \dots + k + (k + 1) = \frac{(k+1)(k+2)}{2}$.

- Rozepíšeme levou stranu a aplikujeme indukční předpoklad (prvních k členů nahradíme zlomkem):
- $\frac{k(k+1)}{2} + (k + 1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2}$.

Závěr

Důkaz je hotov, tvrzení platí pro všechna $n \in \mathbb{N}$.

Princip inkluze a exkluze (PIE)

Účel

Určení počtu prvků sjednocení konečného počtu množin, které se mohou navzájem překrývat (nejsou disjunktní).

Princip

- Sečteme velikosti všech jednotlivých množin.
- Tím jsme ale prvky patřící do průniku dvou množin započítali dvakrát. Proto odečteme velikosti průniků všech dvojic.
- Tím jsme zase odečetli příliš mnoho pro prvky ve třech množinách (byly započteny 3x a pak 3x odečteny). Přičteme průniky všech trojic.
- Znaménka se dále střídají: +, −, +, −, dokud nezahrneme průnik všech množin.

Obecný vzorec

$$|A_1 \cup A_2 \cup \dots \cup A_n| = \sum |A_i| - \sum_{i < j} |A_i \cap A_j| + \sum_{i < j < k} |A_i \cap A_j \cap A_k| - \dots + (-1)^{n+1} |A_1 \cap \dots \cap A_n|$$

Aplikace na příklad (Princip inkluze a exkluze)

Problém: Ve třídě je 30 studentů. 15 studuje angličtinu (A), 20 studuje němčinu (N). 5 nestuduje ani jeden jazyk. Kolik studentů studuje *oba* jazyky současně?

- Celkový počet studentů je 30. Bez obou jazyků je 5. Takže sjednocení studentů s alespoň jedním jazykem: $|A \cup N| = 30 - 5 = 25$.
- Podle PIE pro 2 množiny: $|A \cup N| = |A| + |N| - |A \cap N|$
- Dosadíme: $25 = 15 + 20 - |A \cap N|$
- $25 = 35 - |A \cap N| \implies |A \cap N| = 10$

Závěr

10 studentů studuje oba jazyky současně.

Praktické příklady

- 1. Matematická indukce u smyčky:** Chceme dokázat, že cyklus, který postupně přičítá čísla od 1 do n , po k krocích obsahuje součet $1 + 2 + \dots + k$. Báze platí pro $k = 1$ a indukční krok říká, že když po k krocích máme správný součet, po přičtení $k + 1$ bude správný i pro $k + 1$.
- 2. Inkluze a exkluze v marketingu:** E-shop má 120 zákazníků, 70 použilo slevový kód a 50 nakoupilo přes mobilní aplikaci. Pokud 30 zákazníků splnilo obě podmínky, počet zákazníků v alespoň jedné skupině je $70 + 50 - 30 = 90$.

Algoritmus jako výpočetní model a Churchova teze

Popište algoritmus jako výpočetní model. Co je Churchova teze.

Algoritmus jako výpočetní model

Definice algoritmu

Přesný návod (konečná posloupnost elementárních kroků), jak vyřešit daný typ úlohy.

Vlastnosti algoritmu

Konečnost (Finitnost)

Pro každý platný vstup algoritmus skončí po konečném počtu kroků.

Hromadnost

Neřeší jeden konkrétní příklad (např. $2 + 3$), ale obecnou třídu úloh (např. $a + b$).

Jednoznačnost (Determinismus)

V každém kroku je přesně určeno, co se má stát dál. Za stejných počátečních podmínek vede ke stejným výsledkům.

Rezultativnost

Vždy vydá nějaký výsledek (řešení nebo informaci, že řešení neexistuje).

Výpočetní modely

Abychom mohli o algoritmech dokazovat matematické věty (zda něco jde nebo nejde spočítat), potřebujeme formální model toho, co to znamená "počítat".

Turingův stroj TM

Nejznámější teoretický model počítače.

- Skládá se z nekonečné pásky rozdělené na buňky, čtecí/zápisové hlavy a konečného automatu (řídící jednotky s vnitřními stavy).
- Hlava čte symbol, na základě vnitřního stavu a přečteného symbolu zapíše nový symbol, posune se doleva/doprava a přejde do nového stavu.

Další modely

Lambda kalkul (Alonzo Church), RAM (Random Access Machine - model bližší dnešním procesorům), Postův stroj, Minského stroj.

Ekvivalence modelů

Všechny tyto zmíněné modely mají *stejnou výpočetní sílu*. Co lze spočítat na jednom, lze spočítat i na ostatních (byť s jinou časovou složitostí).

Churchova-Turingova teze (Churchova teze)

Znění

Ke každému intuitivně definovatelnému algoritmu existuje ekvivalentní Turingův stroj, který realizuje tentýž výpočet.

Význam

- Není to matematická věta (nelze formálně dokázat, protože spojuje intuitivní pojem "algoritmus" s formálním pojmem "Turingův stroj").
- Je to **přijímaná hypotéza**.
- Znamená to, že Turingův stroj (a tím pádem i moderní počítač) je **univerzální výpočetní zařízení**. Pokud něco nelze spočítat na Turingově stroji, pak to nelze spočítat žádným algoritmem (v našem vesmíru).

Praktické příklady

1. **Algoritmus v aplikaci:** Přihlášení uživatele je algoritmus: načti e-mail a heslo, ověř formát, najdi uživatele v databázi, porovnej hash hesla a vrať buď platnou session, nebo chybu. Postup je konečný, přesný a pro stejný vstup dává stejný výsledek.
2. **Churchova teze v praxi:** Když úlohu umíme přesně rozepsat jako mechanický postup, bereme ji jako výpočetně realizovatelnou na běžném počítači. Naopak vágní instrukce typu "vyber nejlepší design podle vkusu" není algoritmus, dokud přesně nedefinujeme kritéria a kroky rozhodování.

Rozhodnutelné, nerozhodnutelné, částečně rozhodnutelné problémy a vyčíslitelné funkce

Vysvětlete, co jsou rozhodnutelné, nerozhodnutelné a částečně rozhodnutelné problémy. Vyčíslitelné funkce. Jak se to může projevit v praxi?

Rozhodovací problémy (Problémy ANO/NE)

Jedná se o úlohy, kde pro zadaný vstup očekáváme odpověď z množiny {ANO, NE}. (Např.: "Je dané číslo prvočíslo?", "Existuje v grafu cesta?").

1. Rozhodnutelné problémy

Definice

Problém je rozhodnutelný, pokud existuje algoritmus (Turingův stroj), který pro každý možný vstup v konečném čase skončí a vydá správnou odpověď **ANO** nebo **NE**.

Příklad v praxi

Hledání nejkratší cesty, třídění, testování prvočíselnosti, kompilace správného zdrojového kódu (bez sémantických chyb).

2. Částečně rozhodnutelné problémy

Definice

Problém je částečně rozhodnutelný, pokud existuje algoritmus, který:

- Pro vstupy s odpovědí **ANO** v konečném čase skončí a řekne ANO.
- Pro vstupy s odpovědí **NE** se buď v konečném čase zastaví a řekne NE, nebo **běží donekonečna** (zacyklí se).
- Jinými slovy, pokud je odpověď ANO, určitě se to dozvíme. Pokud je NE, nemusíme se to dozvědět nikdy.

3. Nerozhodnutelné problémy

Definice

Problém je nerozhodnutelný, pokud pro něj **neexistuje** algoritmus, který by jej spolehlivě rozhodl pro každý možný vstup v konečném čase.

Problém zastavení (Halting problem)

Nejslavnější nerozhodnutelný problém (dokázal Alan Turing). Lze napsat program, který vezme jako vstup jiný program P a jeho vstup X , a vždy správně odpoví, zda P na X skončí (ANO) nebo se zacyklí (NE)?

Důkaz (náznak)

Nelze. Pokud by takový program "HALT" existoval, vytvoříme program "PARADOX", který se zeptá HALT. Pokud HALT řekne "skončí", PARADOX se schválně zacyklí. Pokud HALT řekne "zacyklí", PARADOX skončí. To vede k logickému sporu.

Příklady v praxi

- Nalezení dead-locku v libovolném programu.
- Důkaz správnosti libovolného programu (že vždy dělá, co má).
- Antivirové programy (nelze napsat 100% spolehlivý detektor na všechny budoucí viry).

Vyčíslitelné funkce

- Funkce $f(x)$ je vyčíslitelná (computable), pokud existuje algoritmus (Turingův stroj), který pro každý vstup x (pro který je funkce definována) v konečném čase vypočítá hodnotu $f(x)$ a zastaví se.
- Funkce odpovídají rozhodovacím problémům – rozhodovací problém je v podstatě výpočet charakteristické funkce s oborem hodnot $\{0, 1\}$.

Praktické příklady

1. **Rozhodnutelný problém:** Kontrola, zda je číslo sudé, je rozhodnutelná. Algoritmus spočítá $n \bmod 2$ a vždy v konečném čase odpoví ANO nebo NE.
2. **Nerozhodnutelný problém:** Obecný nástroj, který by pro libovolný program a libovolný vstup vždy rozhodl, zda se program někdy zastaví, neexistuje. Proto ani antivir nemůže být teoreticky stoprocentní pro všechny budoucí programy a všechny možné útoky.

Výpočetní složitost problémů, redukce a polynomiální redukce

Vysvětlete výpočetní složitost problémů, redukci a polynomiální redukce.

Výpočetní složitost problémů

- Zkoumá, kolik zdrojů (čas výpočtu, velikost paměti) potřebuje algoritmus v závislosti na velikosti vstupu (n).

Časová složitost $T(n)$

Počet elementárních operací provedených algoritmem pro vstup velikosti n .

Prostorová složitost $S(n)$

Množství paměti potřebné pro běh algoritmu pro vstup velikosti n .

Asymptotická notace (Velké O)

Vyjadřuje chování pro velká n , ignoruje konstanty a pomaleji rostoucí členy. Píše se např. $O(n^2)$, $O(n \log n)$, $O(2^n)$.

Dělení v praxi

Podle růstu složitosti se rozlišuje, zda je problém prakticky řešitelný pro velké vstupy. Nejde jen o konkrétní implementaci, ale o nejlepší známý princip řešení dané třídy úloh.

Traktabilní (zvládnutelné) problémy

Lze je řešit v polynomiálním čase, tj. složitost je $O(n^k)$ pro nějaké fixní k . (Např. třídění, hledání v grafu).

Intraktabilní (nezvládnutelné) problémy

Časová složitost je exponenciální $O(2^n)$, faktoriálová $O(n!)$ apod. S rostoucím vstupem je doba výpočtu astronomická.

Redukce (Převod problémů)

Princip

Redukce problému A na problém B (značíme $A \leq B$) znamená, že umíme vyřešit problém A pomocí černé skříňky (podprogramu), která řeší problém B .

Využití

"Pokud umím řešit B , umím řešit i A ." Z toho vyplývá: Pokud je A těžký problém, pak B musí být alespoň tak těžký jako A (jinak bychom A vyřešili přes B a nebyl by těžký).

- K převodu vstupu z problému A na vstup pro problém B potřebujeme transformační funkci f .
- Vstup x má v problému A odpověď ANO, právě tehdy když $f(x)$ má v problému B odpověď ANO.

Polynomiální redukce

- Redukce je k užítku pouze tehdy, pokud převod (transformace vstupu x na $f(x)$) nezabere sám o sobě více času než samotný výpočet.

Definice

Problém A je polynomiálně redukovatelný na problém B (značíme $A \leq_P B$), pokud existuje algoritmus, který:

1. Transformuje vstup problému A na vstup problému B .
2. Běží v **polynomiálním čase** vzhledem k velikosti původního vstupu.

Význam v teorii složitosti

Pokud $A \leq_P B$ a o problému A víme, že je těžký (nelze ho řešit v polynomiálním čase), pak B také nelze řešit v polynomiálním čase.

- Polynomiální redukce je klíčový nástroj pro dokazování **NP-úplnosti** problémů.

Praktické příklady

1. **Rozdíl složitostí:** Třídění 1 000 prvků algoritmem $O(n \log n)$ je běžně rychlé, protože jde zhruba o tisíce až desetitisíce kroků. Prohledávání všech možností u problému s $O(2^n)$ pro $n = 100$ je prakticky nemožné, protože počet kombinací je astronomický.
2. **Redukce v praxi:** Problém "rozvrhni zkoušky tak, aby student neměl dvě zkoušky současně" lze převést na barvení grafu: zkoušky jsou vrcholy, konflikt mezi dvěma zkouškami je hrana a barva znamená časový slot.

Úplné problémy z hlediska rozhodnutelnosti, NP-úplné problémy

Popište úplné problémy z hlediska rozhodnutelnosti, NP-úplné problémy. Uveďte příklady.

Úplnost problému z hlediska rozhodnutelnosti

Obecný princip úplnosti

Problém je **úplný** pro určitou třídu problémů, pokud:

1. Do této třídy sám patří.
2. Každý jiný problém z dané třídy na něj lze převést vhodnou redukcí.

Úplný problém tedy reprezentuje nejtěžší problémy dané třídy. Pokud bychom našli efektivní algoritmus pro úplný problém, uměli bychom přes redukce řešit i všechny ostatní problémy v této třídě.

V rozhodnutelnosti

U rozhodnutelnosti se redukce používají hlavně k dokazování, že problém rozhodnutelný není. Typickým úplným problémem pro částečně rozhodnutelné problémy je **problém zastavení (Halting problem)**: pro vstupy, kde program zastaví, existuje simulace, která to jednou zjistí, ale pro vstupy, kde program nezastaví, simulace může běžet navždy.

Pokud na nový problém dokážeme převést problém zastavení, ukážeme tím, že nový problém nemůže být obecně rozhodnutelný. Jinak by šlo rozhodnout i problém zastavení, což je spor.

Třídy P a NP (rozhodovací problémy)

Třída P (Polynomiální časy)

Problémy, které dokáže **deterministický** Turingův stroj (běžný počítač) **vyřešit** v polynomiálním čase $O(n^k)$. Jedná se o "prakticky řešitelné" úlohy.

Třída NP (Nedeterministicky polynomiální časy)

Problémy, u kterých, pokud nám někdo předloží **potenciální řešení** (certifikát/nápovědu), jsme schopni toto řešení **ověřit** v polynomiálním čase.

- *Příklad:* Vyřešit Sudoku je těžké (hledání). Ale zkontrolovat vyplněné Sudoku je velmi rychlé (ověření) – to je typická vlastnost NP problémů.

Vztah

Každý problém, který lze rychle vyřešit, lze i rychle ověřit, proto platí $P \subseteq NP$. Největší nevyřešenou otázkou informatiky je, zda $P = NP$.

NP-těžké a NP-úplné problémy (NP-Complete, NPC)

NP-těžký (NP-hard)

Problém H je NP-těžký, pokud se na něj dá **polynomiálně redukovat** každý problém z třídy NP ($X \leq_P H$ pro všechna $X \in NP$). Tj. H je alespoň tak těžký jako všechny problémy v NP. (NP-těžké problémy samotné ale ani nemusí patřit do NP – mohou být ještě těžší, nebo dokonce nerozhodnutelné).

NP-úplný (NP-complete)

Problém C je NP-úplný, pokud:

1. Získal status NP-těžký ($X \leq_P C$ pro všechna $X \in NP$).
2. Zároveň C patří do třídy NP ($C \in NP$).

Znamená to

NP-úplné problémy jsou ty **absolutně nejtěžší problémy v rámci třídy NP**.

Magická vlastnost

Pokud by se komukoliv podařilo najít polynomiální algoritmus (třída P) byť jen pro **jeden jediný** NP-úplný problém, pak by díky polynomiální redukci šly v polynomiálním čase vyřešit *úplně všechny* problémy v NP (dokázali bychom $P = NP$).

Příklady NP-úplných problémů

SAT (Problém splnitelnosti booleovské formule)

První historicky dokázaný NP-úplný problém (Cook-Levinova věta). Máme logickou formuli (např. $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$). Existuje takové ohodnocení proměnných $x_i \in \{0, 1\}$, pro které je celá formule pravdivá (True)?

Problém obchodního cestujícího (TSP - rozhodovací verze)

Máme graf měst s délkami hran a číslo K . Existuje uzavřená cesta, která navštíví každé město právě jednou a její celková délka je $\leq K$?

Problém batohu (Knapsack)

Máme batoh o dané nosnosti a věci s různou hmotností a cenou. Lze do batohu vybrat věci tak, aby se tam vešly, a přitom jejich souhrnná cena byla alespoň C ?

Hledání kliky v grafu

Existuje v daném grafu "klika" (úplný podgraf) o velikosti alespoň k ? (Tj. podskupina k uzlů, kde každý je spojen s každým).

Barvení grafu

Lze vrcholy grafu obarvit maximálně k barvami tak, aby žádné dva sousední vrcholy neměly stejnou barvu?

Praktické příklady

- 1. Sudoku jako intuice NP:** Najít správné vyplnění Sudoku může být těžké, protože je nutné prohledávat mnoho možností. Jakmile ale někdo hotové řešení předloží, ověření řádků, sloupců a bloků je rychlé, což přesně vystihuje myšlenku třídy NP.
- 2. TSP v logistice:** Kurýr má navštívit 20 adres a vrátit se do depa. Ověřit, že konkrétní navržená trasa má délku nejvýše 80 km, je snadné; najít nejlepší trasu mezi všemi možnými pořadími už rychle exploduje počtem kombinací.

Blockchain technologie a kybernetická bezpečnost

Blockchain technologie. Definujte pojmy transaction ledger, blok, transakce, Merkle tree (MT). Definujte, které vlastnosti musí splňovat hashovací funkce, které se používá v MT. Co je to distribuovaný konsenzus? Vyjmenujte základní kybernetické útoky na kryptoměny a jak se jim lze bránit?

Základní pojmy Blockchainu

Blockchain

Distribuovaná, decentralizovaná databáze (účetní kniha), která udržuje neustále rostoucí seznam záznamů (bloků), jež jsou chráněny proti neoprávněnému zásahu zvenčí i ze strany samotných uzlů.

Transaction ledger (Transakční kniha)

Záznam všech proběhlých transakcí. V blockchainu je tato kniha veřejná a sdílená všemi účastníky sítě, což zajišťuje transparentnost.

Transakce

Záznam o přesunu hodnoty (např. kryptoměny) od odesílatele k příjemci. Obsahuje digitální podpis odesílatele (zaručuje autenticitu).

Blok

Datová struktura obsahující hlavičku (s odkazem na hash předchozího bloku) a tělo (seznam potvrzených transakcí). Tvoří jeden "článek" řetězce (chain).

Merkle tree (Merkleův strom)

Binární strom hashů.

- Listy tvoří hashe jednotlivých transakcí v bloku.
- Rodičovské uzly jsou hashe spojení svých potomků.
- Kořen stromu (Merkle Root) je uložen v hlavičce bloku. Zaručuje integritu všech transakcí – změna jediného bitu v jakékoliv transakci lavinovitě změní všechny hashe až po kořen.

Vlastnosti hashovací funkce v Merkle Tree

Kryptografická hashovací funkce (např. SHA-256) musí splňovat:

Odolnost vůči vzoru (Pre-image resistance)

Z hashe nelze (v reálném čase) spočítat původní vstupní data.

Odolnost vůči druhému vzoru (Second pre-image resistance)

K danému vstupu x_1 nelze najít jiný vstup x_2 tak, aby $hash(x_1) = hash(x_2)$.

Odolnost vůči kolizi (Collision resistance)

Nelze najít žádné dva libovolné vstupy $x_1 \neq x_2$ se stejným hashem ($hash(x_1) = hash(x_2)$).

Lavinový efekt

Nepatrná změna vstupu způsobí zcela odlišný výstupní hash.

Distribuovaný konsenzus

Definice

Mechanismus, pomocí kterého se nezávislé uzly v decentralizované síti dohodnou na jediném platném stavu systému (např. na tom, který blok bude jako další přidán do blockchainu), aniž by si musely navzájem důvěřovat.

Příklady algoritmů

Proof of Work (PoW)

Těžaři řeší výpočetně náročný matematický problém (hledání nonce). První úspěšný vytvoří blok.

Proof of Stake (PoS)

Tvůrce bloku je vybrán deterministicky na základě množství vlastněné měny (vkladu/staku).

Kybernetické útoky na kryptoměny a obrana

51% útok (Sybil attack variant)

- *Princip:* Útočník získá více než 50 % celkového výpočetního výkonu sítě (u PoW). Může pak reverzovat své transakce (Double spending) a blokovat ostatní transakce.
- *Obrana:* Růst sítě (čím větší síť, tím je útok nereálnější a dražší), přechod na PoS.

Double spending (Dvojitá útrata)

- *Princip:* Útočník se pokusí odeslat stejné digitální mince dvěma různým příjemcům současně.
- *Obrana:* Počkat na více potvrzení (v Bitcoinu obvykle 6 bloků) před akceptováním platby.

Útoky na chytré kontrakty (Smart Contract exploits)

- *Princip:* Zneužití chyb v kódu (např. Reentrancy útok), které běží nad blockchainem (např. Ethereum).
- *Obrana:* Striktní audit kódu, formální verifikace.

Sybil attack (falešné uzly)

- *Princip:* Útočník vytvoří velké množství falešných identit (uzlů) pro ovlivnění sítě.
- *Obrana:* Právě mechanismy jako PoW/PoS činí tvorbu identit "drahou", čímž útok eliminují.

Praktické příklady

1. **Merkle proof u platby:** Pokud chce peněženka ověřit, že transakce je v konkrétním bloku, nemusí stahovat všechny transakce. Stačí jí transakce, několik hashů ze stromu a Merkle root v hlavičce bloku; z toho ověří, že transakce do bloku patří.
2. **Double spending:** Útočník zaplatí obchodníkovi kryptoměnou a současně se pokusí vytvořit alternativní větev blockchainu, kde stejnou minci pošle sobě. Obchodník se brání tím, že čeká na několik potvrzených bloků, aby bylo přepsání historie ekonomicky nerealné.

Distribuované a paralelní souborové systémy, NFS a GFS

Charakterizujte distribuované souborové systémy. Stručně popište souborový systém NFS, základní operace, implementace, definujte pojem komponentní operace. Charakterizujte paralelní souborové systémy. Stručně popište souborový systém Google File Systém (GFS), algoritmus pro čtení a zápis dat.

Distribuované souborové systémy (DFS)

Charakteristika

Souborový systém, kde klienti, servery a úložná zařízení jsou rozptýleny na různých uzlech sítě. Uživatel se však systém jeví jako jeden lokální souborový systém.

Cíle

Transparentnost (lokace, přístupu), sdílení dat, odolnost proti výpadkům, škálovatelnost.

Souborový systém NFS (Network File System)

Popis

Vyvinut společností Sun Microsystems. Umožňuje připojit (mount) vzdálený adresář ze serveru do lokálního stromu souborů klienta.

Implementace

- Funguje na architektuře Klient-Server.
- Komunikace probíhá typicky přes RPC (Remote Procedure Call).
- Tradičně je NFS server **bezstavový (stateless)** – neudrží si informace o otevřených souborech jednotlivých klientů, což usnadňuje zotavení po výpadku. (Novější NFSv4 stavové prvky přidává).

Základní operace

Vytvoření souboru, čtení, zápis, odstranění, získání atributů.

Komponentní operace (Složené operace v NFSv4)

- Protokol NFSv4 zavedl "Compound operations".
- Namísto posílání mnoha drobných RPC dotazů (např. otevři, přečti, zavři), pošle klient jeden složený (komponentní) dotaz, který obsahuje více operací. Server je postupně vyhodnotí.
- Výrazně se tím snižuje režie a latence na síti.

Paralelní souborové systémy

Charakteristika

Navrženy pro vysoce výkonné počítání (HPC). Data souboru nejsou na jednom disku/serveru, ale jsou "rozprostřena" (striping) napříč více úložnými uzly.

Výhoda

Klienti mohou číst/zapisovat části jednoho obrovského souboru *současně z mnoha serverů* paralelně, čímž se násobí propustnost.

Google File System (GFS)

Popis

Distribuovaný a paralelní souborový systém vyvinutý Googlem pro obrovské objemy dat. Optimalizován pro dávkové čtení a sekvenční zápis (append), běží na levném (commodity) hardwaru, kde se počítá s častými výpadky uzlů.

Architektura

Master node (Řídící uzel)

Udrhuje metadata (adresářovou strukturu, mapování souborů na bloky). Obvykle jen jeden pro jednoduchost.

Chunkservers (Datové uzly)

Ukládají samotná data. Soubory jsou rozděleny na **chunky** (typicky 64 MB). Každý chunk je replikován (běžně 3x) na různé chunkservery.

Algoritmus pro čtení

1. Klient se zeptá Mastera, kde (na kterém chunkserveru) se nachází požadovaný chunk.
2. Master vrátí seznam chunkserverů s replikami.
3. Klient si vybere nejbližší/nejrychlejší chunkserver a přímo od něj data přečte. (Data nejdou přes Mastera!).

Algoritmus pro zápis (Append)

1. Klient získá od Mastera lokace replik chunku. Jedna z replik je určena jako **Primary (Primární)**.
2. Klient pošle data všem replikám najednou (data jsou uložena v bufferu chunkserverů).
3. Klient řekne Primární replice: "Proved' zápis".
4. Primární replika určí přesné pořadí zápisu, aplikuje ho lokálně a přikáže ostatním replikám provést zápis ve *stejném pořadí*.
5. Po potvrzení od všech replik ohlásí Primární replika úspěch klientovi.

Praktické příklady

1. **NFS ve firmě:** Zaměstnanec se přihlásí na libovolnou linuxovou stanici a vidí stejný domovský adresář `/home/novak`, protože je připojený přes NFS ze serveru. Uživatel pracuje lokálně, ale soubory fyzicky

leží na centrálním úložišti.

- 2. GFS pro velké logy:** Služba ukládá terabajty aplikačních logů. GFS rozdělí jeden obrovský soubor na chunky, uloží je na více chunkserverů a každý chunk replikuje, takže lze paralelně číst data a přežít výpadek některého serveru.

Virtualizace: Druhy, hardwarová akcelerace a přístup k hardwaru

Jaké druhy virtualizace znáte. Detailně popište princip hardwarově akcelerované virtualizace. Jak probíhá emulace virtuálního hardwaru? Vysvětlete, jakým způsobem je možné přistupovat ke skutečnému hardwaru z prostředí virtuálního stroje.

Druhy virtualizace

Plná virtualizace (Full virtualization)

Virtuální stroj (VM) beze změny emuluje kompletní hardware. Hostovaný OS netuší, že je virtualizován. Vyžaduje hardwarovou podporu procesoru (Intel VT-x, AMD-V) nebo binární translaci pro zachycení privilegovaných instrukcí.

Paravirtualizace

Hostovaný OS ví, že je virtualizovaný. Jeho jádro je upraveno tak, aby místo volání hardwaru komunikovalo přímo s hypervizorem (Hypercalls). Rychlejší, ale nelze použít pro neupravené OS (např. starší Windows).

Virtualizace na úrovni OS (Kontejnery)

Sdílí jedno jádro (kernel) hostitelského OS. Izolace probíhá na úrovni procesů (v Linuxu pomocí *cgroups* a *namespaces*). Velmi lehká a rychlá (např. Docker).

Hardwarově akcelerovaná virtualizace (Plná virtualizace)

Princip

Tradiční x86 architektura měla problém, že některé citlivé instrukce OS nešly jednoduše zachytit hypervizorem. Hostovaný operační systém přitom musí mít pocit, že běží v privilegovaném režimu a může přímo ovládat CPU, paměť i zařízení.

Hardwarově akcelerovaná virtualizace tento problém řeší tak, že procesor sám rozlišuje běh hypervizoru a běh virtuálního stroje. Výrobci (Intel, AMD) proto přidali instrukční sady **Intel VT-x** a **AMD-V**, které umí bezpečně spustit neupravený OS ve VM a při citlivých operacích automaticky předat řízení hypervizoru.

Jak to funguje (Ring architektura)

- Procesory mají privilegované úrovně (Ring 0 až Ring 3). OS běžně běží v Ring 0, aplikace v Ring 3.
- Hardwarová podpora zavedla nový režim (tzv. **Ring -1** nebo Root/Non-root mode).
- Hypervizor běží v nejvyšší oprávnění (**Root mode**).

- Hostovaný OS běží v Non-root mode, kde má sice "svůj" Ring 0, ale procesor ví, že běží pod hypervizorem.
- Když se VM pokusí o hardwarovou instrukci, procesor to hardwarově zachytí, vyvolá tzv. **VM Exit** a předá řízení hypervizoru. Hypervizor situaci vyřeší a vrátí řízení přes **VM Entry**.

Krokový průběh běhu VM

1. Hypervizor připraví řídicí strukturu VM (např. VMCS/VMCB), kde je uložena konfigurace virtuálního CPU, povolené instrukce, stav registrů a důvody, při kterých má dojít k zachycení.
2. Pomocí **VM Entry** spustí hostovaný OS. Ten běží neupravený a používá vlastní Ring 0, takže se chová stejně jako na fyzickém stroji.
3. Běžné neprivilegované instrukce se vykonávají přímo na procesoru bez zásahu hypervizoru, proto je plná virtualizace s HW podporou výrazně rychlejší než čistá emulace.
4. Citlivé operace (např. změna stránkovacích tabulek, práce s přerušením, I/O instrukce nebo přístup ke speciálním registrům) procesor zachytí jako **VM Exit**.
5. Hypervizor zkontroluje, zda je operace povolená, případně ji emuluje nebo promítne do virtuálního stavu VM, a poté přes **VM Entry** pokračuje v běhu hostovaného OS.

Paměť se dnes typicky řeší pomocí **EPT/NPT** (druhá úroveň stránkování v procesoru), takže CPU hardwarově překládá adresy z pohledu VM na skutečné fyzické adresy hostitele. Tím se zachová izolace VM a zároveň se omezí počet drahých VM Exitů.

Emulace virtuálního hardwaru

- VM potřebuje základní desku, diskový řadič, síťovou kartu.
- O to se stará proces v hostitelském systému (často aplikace typu QEMU).
- Když hostovaný OS zapisuje do "paměti" simulované síťové karty, hypervizor tento přístup zachytí. Emulátor (QEMU) přeloží tento zápis na skutečný síťový požadavek v hostitelském OS. Hostovaný OS vidí standardní (často generický) hardware, ačkoli fyzický server má hardware úplně jiný.

Způsoby přístupu ke skutečnému hardwaru z VM

Softwarová Emulace

Zcela v režii hypervizoru/QEMU. Nejpomalejší (každá operace musí být přeložena).

Paravirtualizované ovladače (např. VirtIO)

Hostovaný OS používá speciální ovladač, který ví o hypervizoru. Nepoužívá emulovaný hardware, ale přes paměť (ring buffery) posílá data rovnou hypervizoru. Mnohem rychlejší.

PCI Passthrough (Device Assignment)

- Umožňuje přiřadit fyzickou PCIe kartu (např. GPU, síťovou kartu) *přímo* do konkrétního virtuálního stroje.
- VM komunikuje se zařízením nativní rychlostí, obchází hypervizor.

- K izolaci paměti zařízení se využívá **IOMMU** (Input-Output Memory Management Unit) na základní desce, která překládá adresy mezi zařízením a VM.

Praktické příklady

1. **Plná virtualizace v datacentru:** Na jednom fyzickém serveru běží hypervizor Proxmox/VMware a na něm několik VM: databázový server, aplikační server a testovací Linux. Každý VM si myslí, že má vlastní disk, síťovou kartu a CPU, ale fyzické zdroje sdílí.
2. **PCI passthrough pro GPU:** Firma má VM pro trénování modelů strojového učení. Aby VM nemusel používat pomalou emulovanou grafiku, hypervizor mu přes IOMMU přímo přiřadí fyzickou GPU kartu a VM ji používá téměř nativní rychlostí.

Stínové stránkování a migrace virtuálních strojů

Vysvětlete princip stínového stránkování a uveďte důvody, proč je nutné tento způsob přístupu do paměti používat. Popište algoritmy pro off-line a on-line migraci virtuálních strojů.

Přístup do paměti a Stínové stránkování (Shadow Paging)

Problém virtualizace paměti

Hostovaný OS (ve VM) očekává spojitý blok fyzické paměti od adresy 0. Ve skutečnosti mu ale hypervizor přiděluje jen část reálné fyzické paměti serveru, která může být navíc fragmentovaná.

Tři úrovně adres

1. Virtuální adresy aplikací ve VM.
2. "Fyzické" adresy hostovaného OS (ve skutečnosti iluze vytvářená hypervizorem).
3. Skutečné fyzické adresy serveru (Machine memory).

Princip stínového stránkování

- Hostovaný OS si udržuje své vlastní tabulky stránek (mapuje úroveň 1 na úroveň 2). Tyto tabulky ale HW procesor nemůže použít.
- Hypervizor proto vytváří a udržuje **Stínové tabulky stránek (Shadow Page Tables)**, které mapují rovnou úroveň 1 na úroveň 3.
- Procesor je nastaven tak, aby pro překlad paměti používal tyto stínové tabulky.

Proč je nutné to používat

- Výkon. Kdyby se každý přístup do paměti musel překládat dvakrát (nejprve tabulkou hostovaného OS a pak tabulkou hypervizoru), bylo by to neúnosně pomalé.
- *Poznámka:* Dnes se toto řeší převážně hardwarově pomocí tzv. **EPT (Extended Page Tables)** / NPT, které dvouúrovňový překlad akcelerují přímo v procesoru, takže softwarové stínové stránkování již není tak nezbytné, ale historicky to byla klíčová technika.

Migrace virtuálních strojů

Přesun běžícího VM z jednoho fyzického serveru na druhý (např. kvůli údržbě, vyvažování zátěže).

1. Off-line migrace (Cold migration)

Princip

VM je pozastaven nebo vypnut, následně se přesune a na cílovém serveru se zapne/obnoví.

Postup

1. Pozastavení chodu VM (Suspend).
2. Zkopírování obsahu operační paměti (RAM) a stavu CPU na cílový server. (Disky obvykle migrovat netřeba, pokud jsou na sdíleném síťovém úložišti - SAN/NAS).
3. Obnovení chodu VM na cílovém serveru.

Nevýhoda

Dlouhý výpadek služby (downtime), což je pro produkční systémy nežádoucí.

2. On-line migrace (Live migration)

Princip

VM běží během přesunu dál, klienti nezaznamenají žádný nebo jen milisekundový výpadek (tzv. brown-out).

Algoritmus (Pre-copy paměti)

Inicializace

Obě strany se dohodnou na migraci (musí mít přístup ke stejnému sdílenému disku).

Iterativní kopírování paměti

Hypervizor začne na pozadí kopírovat RAM z primárního na cílový server, zatímco VM stále běží.

Sledování změn (Dirty pages)

Protože VM stále běží, některé stránky paměti, které už byly zkopírovány, se změní (označí se jako "dirty").

Další iterace

Hypervizor zkopíruje jen tyto "dirty" stránky. Iterace se opakují. V každém kole je špinavých stránek obvykle méně a méně.

Stop-and-copy (Kratičký výpadek)

Když je zbývajících špinavých stránek dostatečně malý, VM se na zlomek sekundy pozastaví, zkopíruje se finální zbytek RAM a stavy CPU registrů.

Přepnutí

VM se okamžitě rozběhne na cílovém serveru, zaktualizují se ARP tabulky na síťových prvcích (aby síťový provoz směřoval na nový server) a na původním serveru se VM zruší.

Praktické příklady

1. **Překlad paměti ve VM:** Aplikace ve VM pracuje s virtuální adresou `0x1000`. Hostovaný OS ji přeloží na svou "fyzickou" adresu, ale hypervizor ji musí namapovat na skutečnou fyzickou RAM serveru;

stínové stránkování nebo EPT zajistí, aby CPU našel správné místo rychle a izolovaně.

- 2. Live migrace při údržbě:** Administrátor potřebuje vyměnit paměťový modul ve fyzickém serveru. Běžící VM s e-shopem přesune live migrací na jiný hostitel, zákazníci zaznamenají nanejvýš krátké zpomalení a server lze bezpečně vypnout.

Zajištění vysoké dostupnosti a Virtual Network Embedding

Popište detailně dva základní principy, které se používají v oblasti virtualizace pro zajištění vysoké dostupnosti (vytváření snímků a asynchronní replikace). Definujte problém Virtual Network Embedding a způsoby jeho řešení.

Zajištění vysoké dostupnosti (High Availability - HA) ve virtualizaci

Cílem HA je minimalizovat výpadky v případě selhání hardwaru nebo softwaru. Základními principy jsou zálohování stavu a dat.

1. Vytváření snímků (Snapshots)

Princip

Snímek zachycuje přesný stav virtuálního stroje v daném okamžiku. Zahrnuje:

Stav disku

Soubory na disku VM se zmrazí. Nové zápisy se začnou ukládat do nového souboru (tzv. delta soubor nebo diferenciální disk). Původní disk je read-only.

Stav paměti (volitelně)

Uložení kompletního obsahu RAM na disk.

Konfiguraci VM

Počet CPU, síť atd.

Využití

Okamžitý návrat v čase (Rollback) při neúspěšné aktualizaci systému nebo poškození dat. Není to plnohodnotná záloha proti fyzickému zničení úložiště, ale klíčový nástroj pro dostupnost aplikačního běhu.

2. Asynchronní replikace (Disaster Recovery)

Princip

Neustálé (nebo v pravidelných intervalech probíhající) kopírování dat VM na sekundární (často geograficky vzdálené) úložiště/lokalitu.

Jak to funguje

- "Asynchronní" znamená, že když VM zapíše data na disk, zápis je potvrzen lokálně a VM pokračuje ve výpočtu. Teprve následně (na pozadí) se tato změna odešle po síti do záložní lokality.

- Oproti synchronní replikaci nemá negativní dopad na výkon primárního VM (nezdržuje zápisy kvůli latenci sítě).

Využití

Při katastrofickém výpadku celého datacentra lze VM nainstalovat v sekundární lokalitě. Může dojít k nepatrné ztrátě nejnovějších dat ($RPO > 0$), ale systém je velmi rychle dostupný.

Virtual Network Embedding (VNE)

Kontext

V cloudovém prostředí chtějí zákazníci vytvářet vlastní virtuální sítě (Virtual Networks - VN) s konkrétní topologií (virtuální routery, switchy, linky s garantovanou propustností).

Problém VNE

Jde o optimalizační problém, jak efektivně "vložit" (namapovat/naembedovat) tyto virtuální sítě na *sdílenou fyzickou infrastrukturu* (Substrate Network) poskytovatele.

Node mapping

Každý virtuální uzel musí být umístěn na fyzický server, který má dostatek kapacity (CPU, RAM).

Link mapping

Každá virtuální síťová linka musí být namapována na jednu nebo více fyzických tras v síti tak, aby byla dodržena požadovaná šířka pásma a zpoždění (latence).

Způsoby řešení

- Je to **NP-těžký** problém (varianty problému batohu a hledání cest).
- V praxi se řeší pomocí **heuristických a meta-heuristických algoritmů** (např. genetické algoritmy, simulované žíhání, greedy algoritmy), které najdou "dostatečně dobré" řešení v reálném čase.
- Typický greedy přístup: Nejdříve namapuj uzly, které požadují nejvíce prostředků, a pak hledej nejkratší cesty pro linky s využitím např. Dijkstrova algoritmu nebo K-nejkratších cest.

Praktické příklady

1. **Snapshot před aktualizací:** Správce před instalací nové verze účetního systému vytvoří snapshot VM. Pokud aktualizace poškodí aplikaci, vrátí VM do původního stavu během minut místo obnovy celého serveru ze zálohy.
2. **VNE u cloudového zákazníka:** Zákazník chce virtuální síť se třemi VM, firewalllem a garantovanou propustností 1 Gb/s mezi databází a aplikací. Poskytovatel musí rozhodnout, na které fyzické servery VM umístí a kudy povede fyzická síťová trasa, aby splnila kapacitu i latenci.

SDN, NFV, SDWAN a VXLAN

Definujte architekturu a princip SDN, strukturu SDN zařízení, integrace protokolu OpenFlow do SDN. Dále pojmy SDWAN, NFV a VNF. Popište princip a význam technologie VXLAN.

SDN (Software Defined Networking)

Princip

Tradiční síťové prvky (switche, routery) v sobě mísí inteligenci rozhodování (Control plane) a samotné předávání paketů (Data plane). SDN tyto dvě roviny **striktně odděluje**.

Architektura SDN

Aplikační vrstva

Programy, které definují, jak se má síť chovat (bezpečnostní politiky, load balancing).

Řídící vrstva (Control plane)

Centralizovaný **SDN Controller**. "Mozek" sítě. Přijímá příkazy z aplikací a převádí je na pravidla pro síťové prvky. Vidí celou topologii sítě.

Datová vrstva (Data plane / Infrastructure)

"Hloupé" fyzické nebo virtuální switche, které pouze rychle předávají pakety na základě tabulek, jež jim poslal controller.

Integrace protokolu OpenFlow

- OpenFlow je standardní komunikační protokol (Southbound API) mezi SDN Contollerem a datovou vrstvou (switchi).
- Umožňuje controlleru vzdáleně zapisovat a modifikovat "flow tabulky" ve switchích. Tyto tabulky určují, co se stane s konkrétním paketem (přepošli na port X, zahod', uprav hlavičku).

NFV a SDWAN

NFV (Network Functions Virtualization)

Princip

Přesun síťových funkcí (firewally, NAT, load balancery, IDS), které dříve běžely na specializovaném (drahém) hardwaru, do podoby softwaru.

VNF (Virtual Network Function)

Samotná softwarová implementace síťové funkce (např. virtuální firewall běžící jako VM na běžném serveru).

Vztah k SDN

SDN odděluje řízení od dat, NFV nahrazuje dedikovaný hardware softwarem. Vzájemně se doplňují (SDN může směřovat provoz do VNF instancí).

SDWAN (Software-Defined Wide Area Network)

- Aplikace principů SDN na rozlehlé sítě (WAN) spojující pobočky firem a datacentra.
- Odděluje řízení sítě od hardwaru linek (MPLS, širokopásmový internet, LTE/5G).
- Centralizovaně směřuje provoz podle typu aplikace přes nejvhodnější linku (např. videohovor přes kvalitní linku, běžný web přes levný internet).

VXLAN (Virtual Extensible LAN)

Problém tradičních VLAN

Omezení na 4096 identifikátorů (kvůli 12bitové hlavičce). Nepostačující pro masivní cloudová datacentra s desítkami tisíc zákazníků (tenantů). Dále neschopnost protáhnout L2 síť (VLAN) přes L3 routery.

Princip VXLAN

Technologie zapouzdření (encapsulation) typu **MAC-in-UDP**.

- Bere standardní L2 Ethernetový rámec a celý ho zabalí do L4 UDP datagramu, který se odešle přes L3 IP síť.

Význam

- Rozšiřuje počet virtuálních sítí na 16 milionů (24bitový VXLAN Network Identifier - VNI).
- Umožňuje roztáhnout L2 podsíť (Layer 2) přes rozlehlou L3 infrastrukturu. Pro VM to vypadá, že jsou na stejném lokálním switchi, i když fyzicky mohou být na opačném konci datacentra.

Praktické příklady

1. **SDN v kampusové síti:** Univerzita má centrální SDN controller, který při připojení studenta automaticky nastaví pravidla ve switchích: student jde do studentské VLAN, zaměstnanec do interní sítě a neznámé zařízení do izolované sítě.
2. **VXLAN v cloudu:** Dva virtuální stroje jednoho zákazníka běží v různých serverovnách, ale mají být ve stejné L2 síti. VXLAN zabalí jejich Ethernet rámce do UDP paketů a přenesení je přes L3 síť tak, že VM vidí stejnou lokální síť.

Cloud Computing: Distribuční a platební modely, základní pojmy

Distribuční model se zabývá tím, co je v rámci cloudové služby nabízeno. Vysvětlete termíny: PaaS, IaaS, SaaS. Jaký je nejrozšířenější platební model v oblasti cloudových služeb (platba za spotřebovaný výkon/paměť/přenos). Vysvětlete tyto pojmy v kontextu cloud computingu: multitenantnost, škálovatelnost, On-demand self-service, Resource pooling, Rapid elasticity, Measured service

Distribuční modely (Co cloud nabízí)

Určují míru odpovědnosti mezi poskytovatelem cloudu a zákazníkem.

IaaS (Infrastructure as a Service - Infrastruktura jako služba)

Nabízí

Surové výpočetní prostředky (Virtuální stroje, diskový prostor, virtuální sítě).

Odpovědnost zákazníka

Správa operačního systému (aktualizace, bezpečnost), middleware, runtime a samotná aplikace.

Příklady

Amazon EC2, Microsoft Azure Virtual Machines.

PaaS (Platform as a Service - Platforma jako služba)

Nabízí

Prostředí (platformu) pro nasazení aplikací, včetně OS, databází, webových serverů a vývojářských nástrojů.

Odpovědnost zákazníka

Vyvíjí a spravuje pouze svá data a kód aplikace. Nemusí se starat o správu OS nebo záplatování serveru.

Příklady

Heroku, Google App Engine.

SaaS (Software as a Service - Software jako služba)

Nabízí

Hotovou aplikaci přístupnou přes internet (nejčastěji webový prohlížeč).

Odpovědnost zákazníka

Žádná správa infrastruktury ani kódu, pouze běžné uživatelské nastavení a data.

Příklady

Google Workspace (Docs, Gmail), Microsoft Office 365, Salesforce.

Platební model

Nejrozšířenější model

Pay-as-you-go (Platba za použití).

- Zákazník neplatí fixní paušály, ale přesně účtované peníze za reálně spotřebované zdroje (hodiny běhu CPU, GB obsazeného prostoru na disku, GB přenesených dat po síti).
- Změna IT z kapitálových výdajů (CapEx - nákup serverů) na provozní výdaje (OpEx).

Základní charakteristiky Cloud Computingu

Multitenantnost (Multi-tenancy)

Schopnost cloudu sloužit více zákazníkům (tenantům) současně ze sdílené fyzické infrastruktury, přičemž jsou jejich data a výpočty navzájem bezpečně izolovány a odděleny.

On-demand self-service (Samoobsluha na vyžádání)

Zákazník si může sám, automatizovaně a okamžitě (přes webový portál nebo API) zřídit výpočetní zdroje, aniž by musel komunikovat s lidskou podporou poskytovatele.

Resource pooling (Sdružování prostředků)

Poskytovatel sdružuje fyzické zdroje (CPU, RAM, disky) do velkých "bazénů" (poolů), ze kterých dynamicky přiděluje kapacitu zákazníkům podle jejich aktuální potřeby (často využívá virtualizaci).

Rapid elasticity (Rychlá pružnost/škálovatelnost)

Schopnost systému rychle (často automaticky - autoscaling) navyšovat (scale out/up) i snižovat (scale in/down) poskytované zdroje přesně podle aktuální zátěže (např. e-shop přidá servery během Black Friday a pak je zase smaže). Pro zákazníka se kapacita jeví jako nekonečná.

Measured service (Měřená služba)

Systém automaticky měří a monitoruje využití zdrojů. Tato transparentnost je nutná pro Pay-as-you-go platební model i pro optimalizaci a plánování kapacit.

Praktické příklady

1. **IaaS vs. SaaS:** Když si firma pronajme virtuální server v AWS EC2 a sama instaluje Linux, databázi a aplikaci, používá IaaS. Když stejná firma používá Gmail nebo Microsoft 365 bez správy serverů,

používá SaaS.

2. **Pay-as-you-go u e-shopu:** Během Black Friday e-shop automaticky navýší počet serverů z 3 na 20. Po kampani je zase sníží zpět na 3 a zaplatí jen hodiny, po které skutečně využíval vyšší kapacitu.